PUCP

PROGRAMACIÓN 3 (1INF30)

Introducción a JAVA y a la JVM

2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi

AGENDA

1. Definición y Propósito de JAVA
2. Historia y Origen de JAVA
3. Lenguajes Compilados Nativamente
4. Lenguajes Interpretados
5. El Modelo Híbrido de Java
6. Diferencias entre JDK y JRE
7. El Compilador de JAVA
8. El Intérprete de JAVA
9. Variables de Entorno en JAVA
10. Configuración de las Variables de Entorno
11. Java - JIT (Just In Time Compilation)

AGENDA

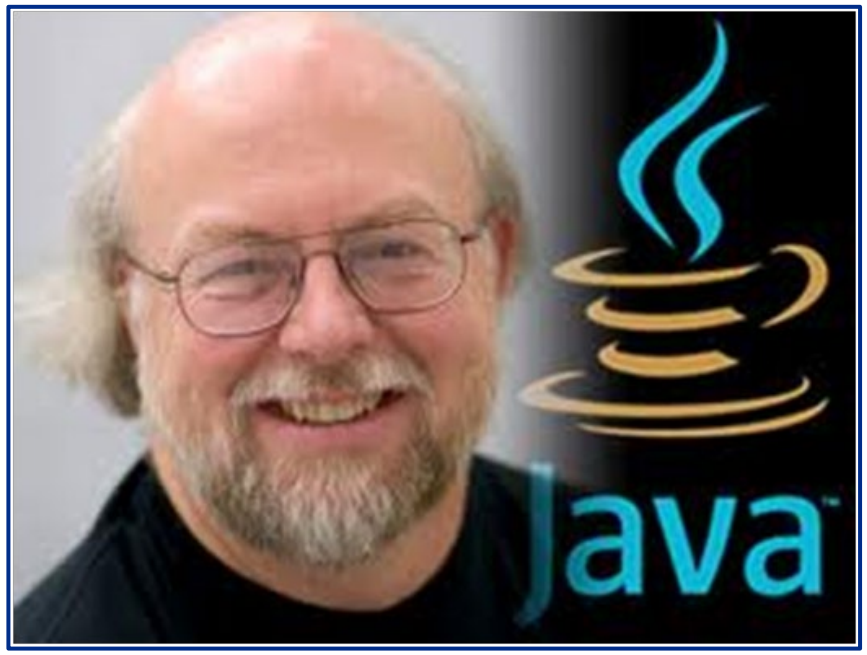
12. La Máquina Virtual de JAVA
13. Estructura de la JVM
14. Recolección de Basura
15. Ventajas de la Gestión Automática de Memoria
16. Limitaciones de la Gestión Automática de Memoria
17. Configuración de la Variable de Entorno PERMANENTE
18. Configuración de la Variable de Entorno TEMPORAL
19. Codificando, Compilando y Ejecutando el primer programa

Definición y Propósito de JAVA

- **JAVA** es un lenguaje de programación diseñado para la portabilidad y la independencia de la plataforma.
- Su arquitectura se basa en el principio "***write once, run anywhere***" (escribir una vez, ejecutar en cualquier lugar).
- Utiliza una Máquina Virtual (**JAVA Virtual Machine**) para aislar el software del hardware subyacente.

Historia y Origen de JAVA

- Creación: El proyecto fue iniciado por **James Gosling** en **Sun Microsystems** bajo el nombre "**Oak**", diseñado originalmente para dispositivos electrónicos de consumo.



Historia y Origen de JAVA

- **1995 - Lanzamiento:** Se presentó oficialmente como Java 1.0, introduciendo la **Java Virtual Machine** y el principio de portabilidad para la ejecución en múltiples plataformas.

- **2010 - Adquisición: Oracle Corporation** adquirió **Sun Microsystems**, asumiendo desde entonces el desarrollo y la gestión de los estándares de la plataforma Java.

Lenguajes Compilados Nativamente

- Lenguajes como **C++** o **Golang** se transforman directamente en **código de máquina** por un **compilador**.
- El código resultante es específico para una arquitectura de hardware y sistema operativo.
- Estos lenguajes suelen ofrecer una alta eficiencia de ejecución a costa de la portabilidad.

Lenguajes Interpretados

- Lenguajes como **Python** o **JavaScript** ejecutan el código fuente mediante **un intérprete** durante **el tiempo de ejecución**.
- La interpretación permite la ejecución en múltiples plataformas sin recompilación manual.
- Las implementaciones modernas suelen incorporar técnicas de compilación **Just-In-Time** (**JIT**).

El Modelo Híbrido de JAVA

- **JAVA** se define como un lenguaje híbrido al ser simultáneamente **compilado** e **interpretado**.
- El **código fuente** se compila primero en un formato intermedio denominado **Bytecode**.
- Posteriormente, la Java Virtual Machine (**JVM**) **interpreta** o **compila** este Bytecode según sea necesario en el entorno de destino.

Diferencias entre JDK y JRE

- **JRE (Java Runtime Environment):** Es el entorno de ejecución que contiene la **Java Virtual Machine** (**JVM**) y las bibliotecas estándar necesarias para **ejecutar** aplicaciones Java (**java**).
- **JDK (Java Development Kit):** Es el kit de desarrollo completo que incluye el JRE más herramientas específicas para programadores, como el compilador (**javac**).
- El **JRE** está incluido dentro del **JDK**; no es posible compilar código fuente (**.java**) utilizando únicamente el **JRE**.
- El **JRE** está destinado al usuario final que solo requiere ejecutar software, mientras que el **JDK** es indispensable para el desarrollo y mantenimiento de sistemas.

El Compilador de JAVA (javac)

- El programa **`javac`** es la herramienta encargada de transformar archivos **`.java`** en archivos **`.class`**.
- Los archivos **`.class`** contienen **Bytecode**, un conjunto de instrucciones neutrales respecto a la plataforma.

```
# javac Programa.java
```

El Intérprete de JAVA (java)

- El programa **`java`** actúa como el lanzador de la máquina virtual que carga y ejecuta los archivos **`.class`**.
- El intérprete traduce el **Bytecode** en instrucciones que el sistema operativo puede procesar.

```
# java -cp . Programa.class
```

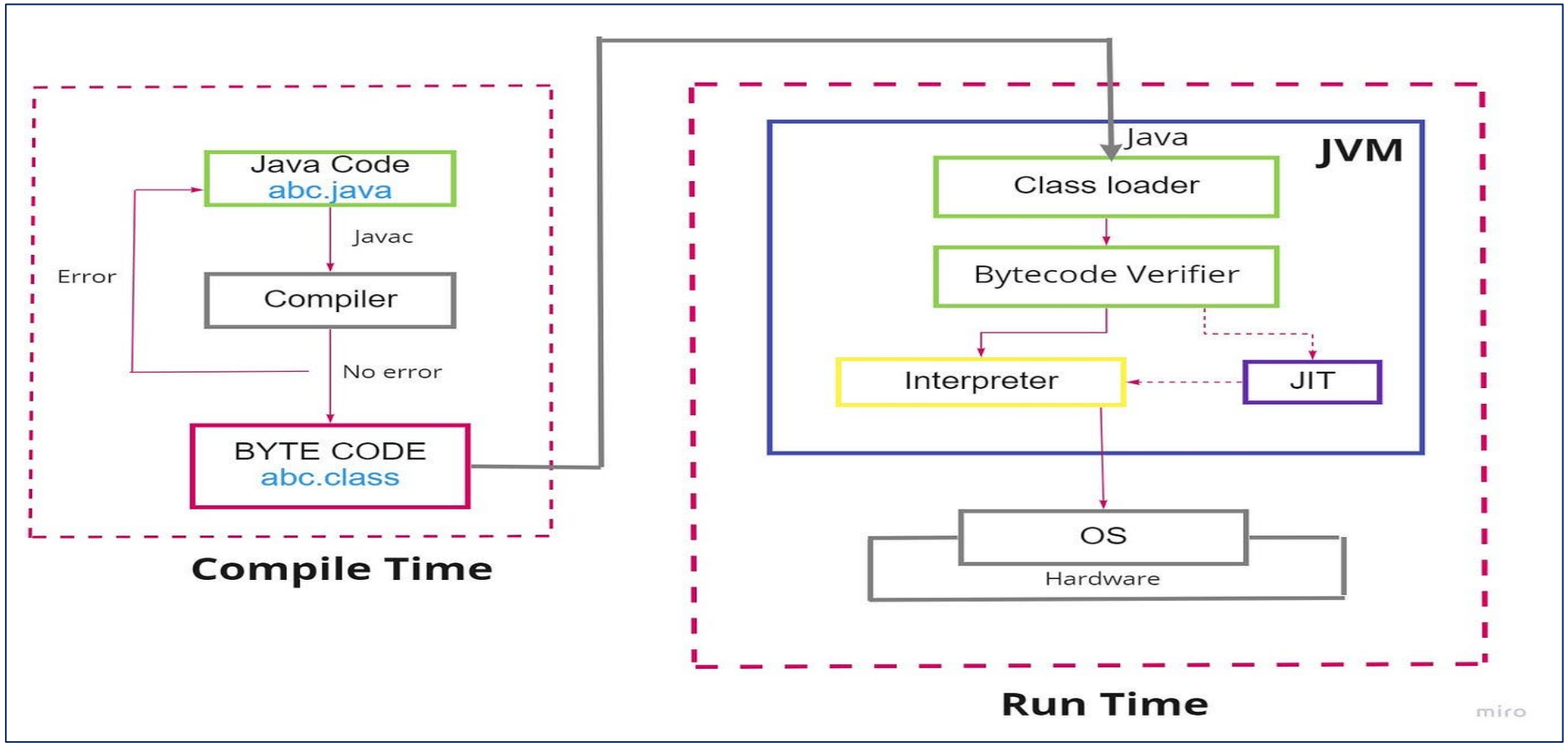
Variables de Entorno en JAVA

- **JAVA_HOME:** Especifica la ruta del directorio principal donde se encuentra instalado el Java Development Kit (JDK).
- **PATH:** Incluye la ruta al directorio **bin** del JDK, permitiendo la ejecución de los comandos **javac** y java desde cualquier ubicación del sistema.
- **CLASSPATH:** Determina las rutas donde la **Java Virtual Machine** (**JVM**) debe buscar las bibliotecas y los archivos .class necesarios para la ejecución del programa.

Configuración de las Variables de Entorno

- **Configuración en Windows (CMD)**
  - Variable JAVA_HOME: `set JAVA_HOME=C:\Program Files\Java\jdk-25`
  - Actualización del PATH: `set PATH=%JAVA_HOME%\bin;%PATH%`
  - Verificación: `java -version` (debe retornar la referencia a la versión 25).
  - Persistencia: Para que los cambios en el JDK 25 sean permanentes, las variables de entorno se deben modificar en el Panel de Control.
- **Configuración en Linux / MacOSX**
  - Comando EXPORT: `export JAVA_HOME=/usr/lib/jvm/java-25`
  - Actualización del PATH: `export PATH=$JAVA_HOME/bin:$PATH`
  - Persistencia: Para que los cambios en el JDK 25 sean permanentes, los comandos deben incluirse en el archivo .bashrc o .zshrc.

Java - JIT (Just In Time Compilation)



Fuente: [Understanding Just-In-Time (JIT) Compilation in Java | by Sakshee Agrawal | Medium]

La Máquina Virtual de JAVA

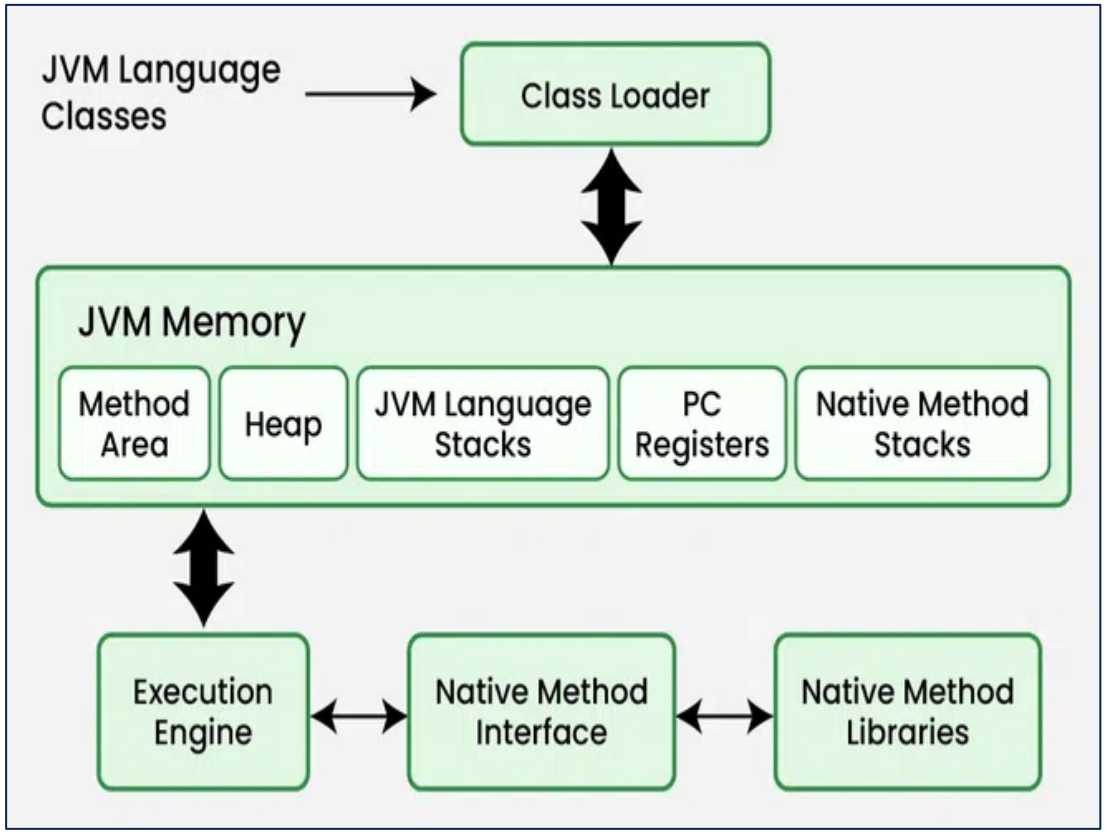
- La **JVM** es el entorno de ejecución que procesa el **Bytecode** de **Java**.
- Actúa como un intermediario entre el código compilado y el sistema operativo.
- Permite que el **Bytecode** sea neutral respecto a la plataforma física.

Nota: La JVM no es neutral respecto a la plataforma.

- La JVM depende del sistema operativo y la arquitectura.
- No es la misma **JVM** en Windows, Linux o Mac.

Estructura de la JVM

- La arquitectura interna se compone de tres subsistemas: Class Loader, JVM Memory y Execution Engine.
- El Class Loader es responsable de la carga de las clases necesarias para la ejecución.
- El motor de ejecución procesa las instrucciones contenidas en el Bytecode.



Recolección de Basura (Garbage Collector)

- El **Garbage Collector** rastrea cada objeto en la memoria **Heap** para determinar su uso.
- Identifica y marca los objetos que están activos y aquellos que ya no son referenciados.
- Remueve automáticamente los objetos marcados como no usados para liberar recursos.

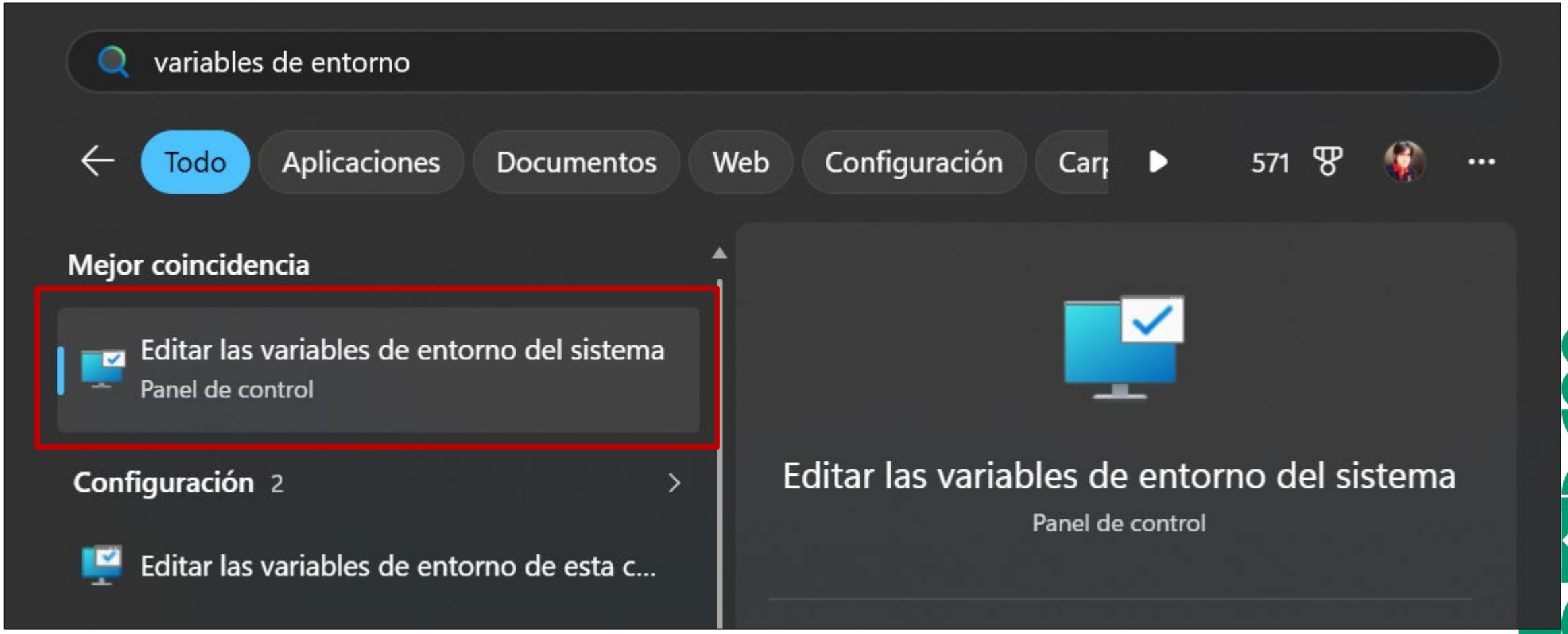
Ventajas de la Gestión Automática de Memoria

- Elimina la necesidad de realizar la asignación y liberación de memoria de forma manual.
- Reduce la incidencia de errores relacionados con la gestión de punteros nulos.
- Permite una gestión automática de las fugas de memoria (**Memory Leaks**).

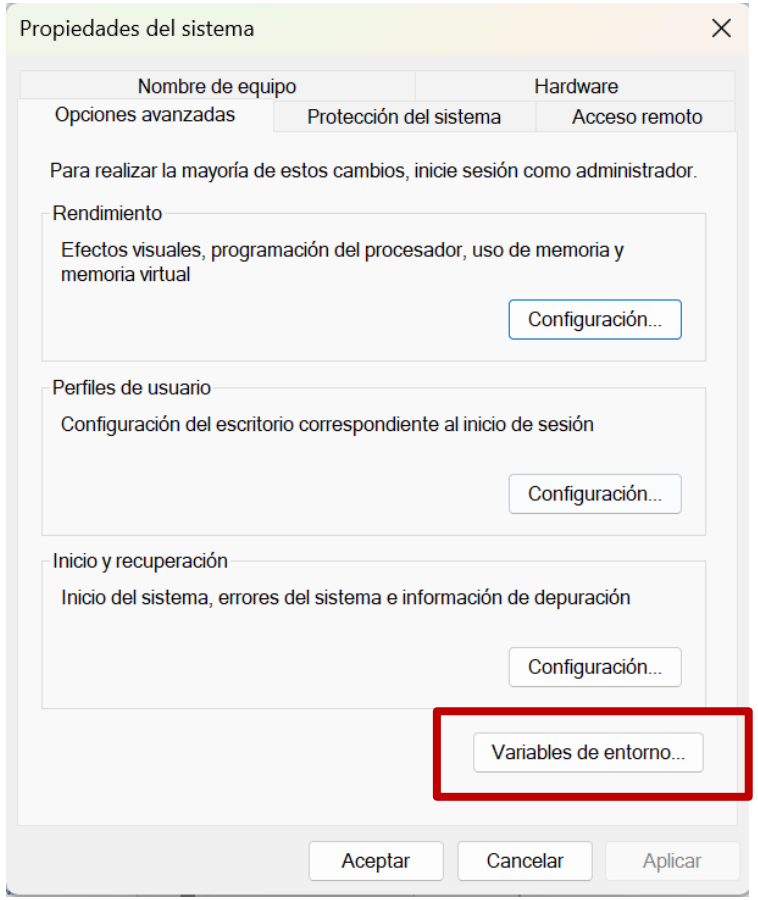
Limitaciones de la Gestión Automática de Memoria

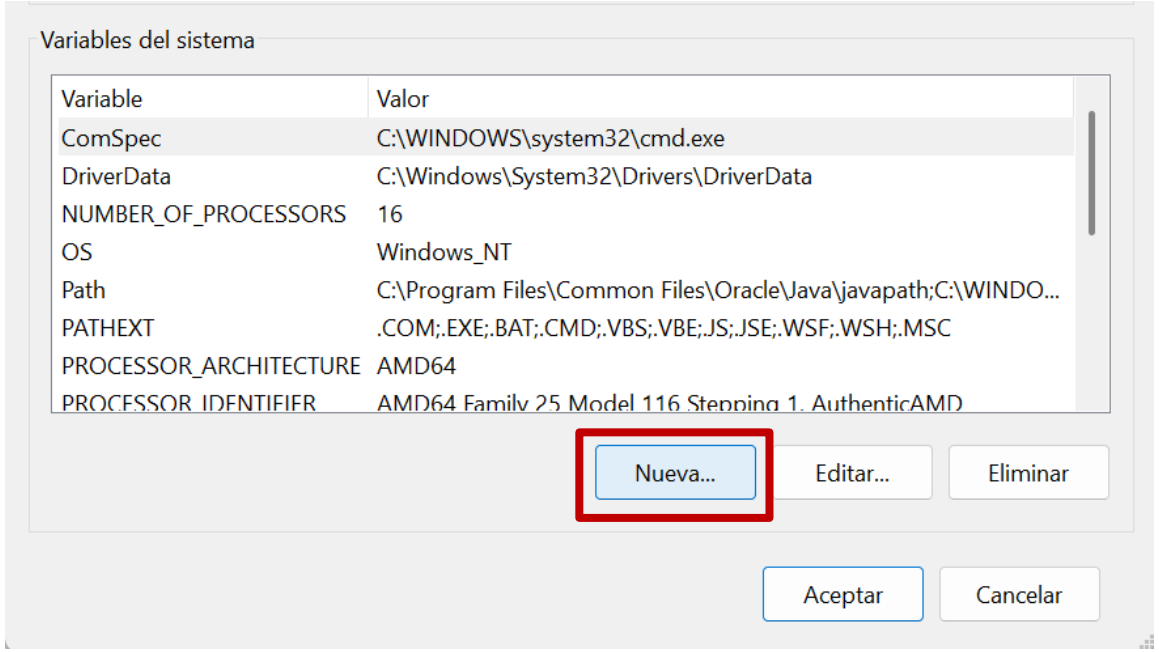
- El proceso de recolección de basura requiere ciclos adicionales de CPU.
- Los desarrolladores poseen un control limitado sobre la administración directa de la memoria.
- La ejecución del recolector puede afectar temporalmente el rendimiento de la aplicación.

Configuración de la Variable de Entorno Permanente

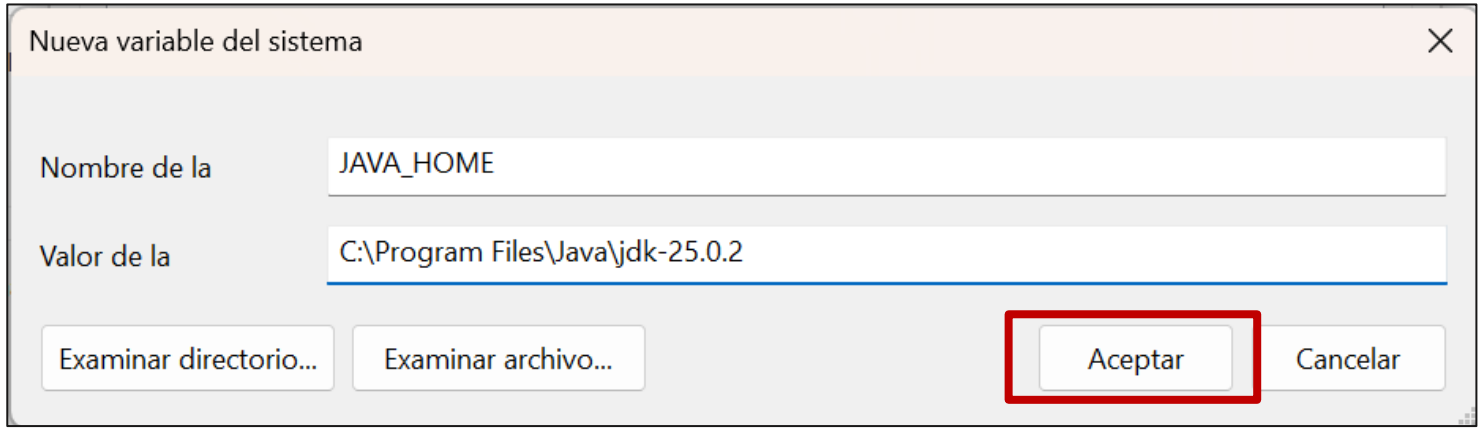


Configuración de la Variable de Entorno Permanente

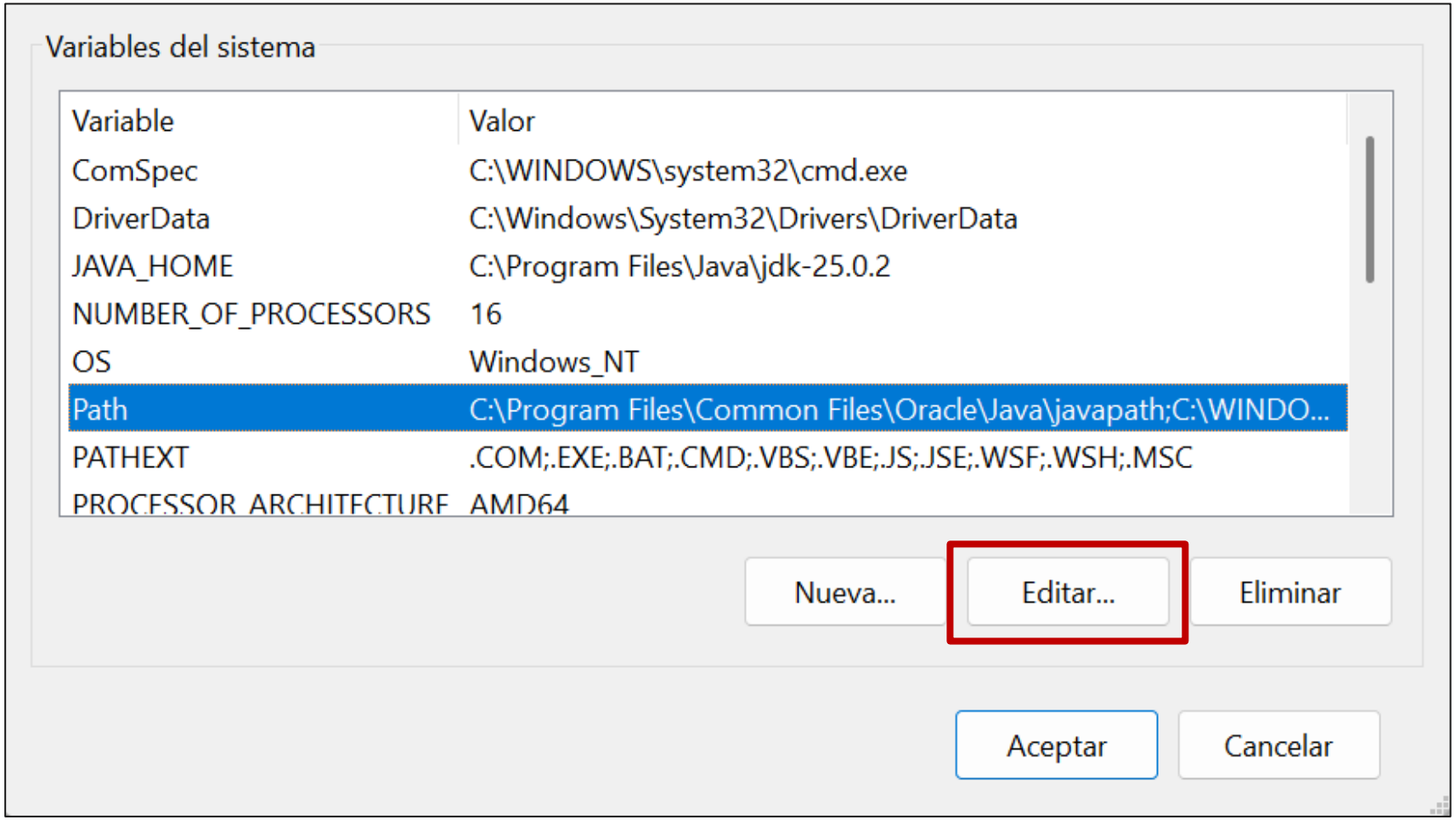




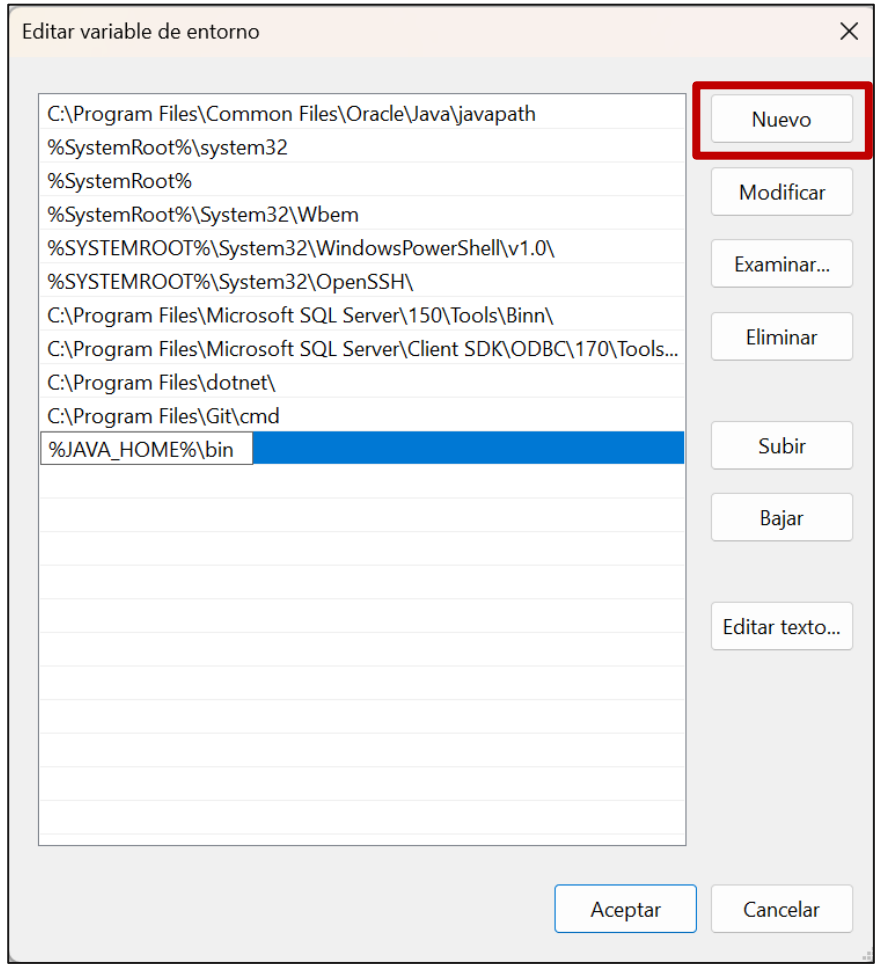
Configuración de la Variable de Entorno Permanente

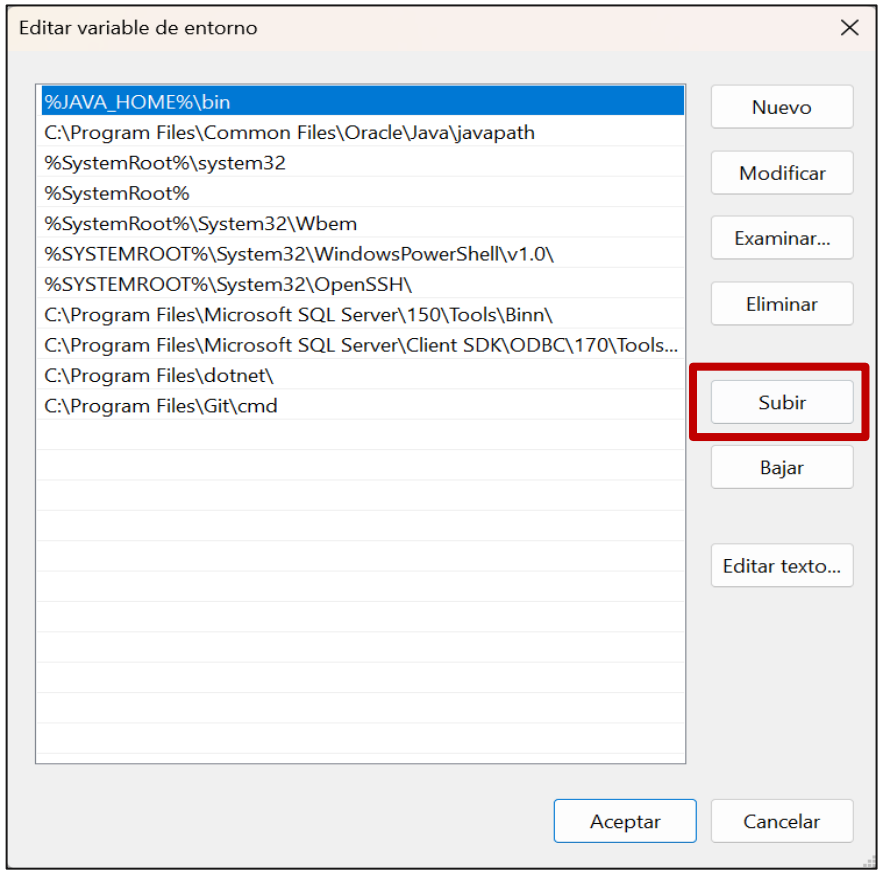


Configuración de la Variable de Entorno Permanente



Configuración de la Variable de Entorno Permanente





- Finalmente, aceptar todo.

Configuración de la Variable de Entorno Permanente

Símbolo del sistema

```
C:\>java -version
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)

C:\>
```

Configuración de la Variable de Entorno Temporal

Símbolo del sistema

```
C:\>SET JAVA_HOME=C:\Program Files\Java\jdk-25.0.2

C:\>SET PATH=%JAVA_HOME%\bin;%PATH%

C:\>java -version
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)

C:\>
```

Codificando, Compilando y Ejecutando el primer programa

Ejemplo.java

```java
class Ejemplo{
    public static void main(String[] args){
        System.out.println("Hola Mundo...");
    }
}
```

Símbolo del sistema

```
C:\Prog3>javac -cp . Ejemplo.java

C:\Prog3>java Ejemplo
Hola Mundo...

C:\Prog3>
```

¡Gracias!

PUCP